

Experiment 09

Name	: Johnson Kumar	UID	: 23BCS12654
Branch	: CSE	Section	: KRG-1A
Semester	: Sixth	Date of exp.	: 13/04/2026
Subject	: System Design	Subject Code	: 23CSH-314

AIM

To design a Ticket-Booking platform

It's a platform that allows users to discover and purchase tickets for various entertainment options like movies, concerts, and live events.

OBJECTIVE:

- Enable users to discover events using search filters (title, location, date)
- View detailed event and seat information
- Select and book tickets securely without conflicts.

PROCEDURE

- Understood the problem and identified the system's goal and users.
- Listed all the functional requirements the system needed to perform.
- Defined the non-functional requirements such as scale, availability, and consistency.
- Estimated the capacity including users, traffic, and storage needs.
- Identified the core entities and designed the basic data model.
- Designed the APIs with endpoints and request/response formats.
- Identified potential bottlenecks and critical challenges.
- Decided key design approaches like caching, locking, and asynchronous processing.
- Chose appropriate databases and storage technologies.

Core-Entities of the System:

- | | | |
|---------|-----------|------------|
| - Users | - Event | -Seat Lock |
| - Venue | - Booking | |
| - Seats | - Payment | |

FUNCTIONAL REQUIREMENTS

- a) Ability to search events by title, location, and date
- b) Ability to view detailed information of an event including seats and metadata
- c) Ability to select seats for a specific show
- d) Ability to book tickets by completing a transaction
- e) Ability to view booking details/history

NON-FUNCTIONAL REQUIREMENTS

- a) Scale: 10M daily active users
- b) CAP Theorem: Consistency & Availability Trade-offs
 - i. Maintained strong consistency during ticket booking to prevent double booking.
 - ii. Ensured high availability for search and event viewing.
- c) Latency: 20-50ms
- d) Maintained fault tolerance and reliability under failures
- e) Ensured security for user data and payment processing

API DESIGN

Search APIs : GET /events?title=&city=&date=
Event Details : GET /events/{event_id}
Seat Availability : GET /events/{event_id}/seats
Lock Seats (Important for consistency)
: POST /seats/lock
Body: { event_id, seat_ids, user_id }
Book Ticket : POST /bookings
Body: { user_id, event_id, seat_ids, payment_id }
User Bookings : GET /users/{user_id}/bookings

HIGH-LEVEL DESIGN

For reference please view the attached stock-trade.io file on draw.io(diagrams.net).

LEARNING OUTCOMES

- Understood how to design a large-scale distributed system for real-world applications
- Learned to differentiate and apply functional and non-functional requirements
- Gained knowledge of scalability, availability, and consistency trade-offs (CAP principles)
- Learned how to prevent race conditions using locking mechanisms like Redis seat locks
- Understood the importance of caching and search optimization for read-heavy systems
- Gained experience in designing microservices and their interactions
- Learned how to design APIs and structure data flow between services
- Understood database selection strategies (SQL vs NoSQL vs cache)
- Learned how to handle high traffic and ensure fault tolerance
- Gained insight into real-world challenges like payment handling and asynchronous processing